Catalogue extraction guide

Every supplier site is built differently, so there is one scraper per site and one **shared exporter**. Scrapers only have to turn a site's data into `Product` / `Variant` objects (`sp_scraper/models.py`); everything after that is common.

```
site  ->  <site>_scraper  ->  Product/Variant  ->  sp_scraper/exporters.py  ->  TBI Excel + Shopify CSV
```

1. Output rules (apply to every supplier)

Rule	Detail
Catalogue file	`*_catalogue_all.xlsx` , the 29 TBI columns, **one row per sellable variant** (colour + size)
Shopify file	`*_shopify_products_all.csv` , one Shopify product per style, options Color / Size, 2.5x markup on wholesale, status `draft`
Style rule	`StyleCode` identifies the **style without colour**. Colours are rows / options inside the style. Never put the colour in `StyleCode` (the exporter strips a `NNNNN-NNN` colour suffix via `base_style_code`)
`ItemCode`	Unique per variant (style + colour + size). The check is `distinct ItemCode == rows`
Prices	`Wholesale_Price_Euro` = your buying price, `Retail_Price_Euro` = RRP / UVP, decimal comma
De-duplication	Dedupe on product URL while crawling and on article number after scraping. Never export the same item twice
Resume	Each scraper appends to a `*_products.jsonl` ; re-running skips finished items, `--fresh` starts over
Secrets	Cookies, passwords and sessions live in `.auth/` or `.env` , both gitignored. Never commit them

Count definitions used in reports: **styles** (what the supplier shows), **colour items** (a style in one colour), **variants / rows** (colour x size).

2. Supplier playbooks

	Sisters Point	Bestseller (MORE)	Hailys
Package	`sp_scraper`	`bs_scraper`	`hl_scraper`
URL	b2b.sisterspoint.com	more.bestseller.com	shop.hailys-fashion.com
Platform	Sumoshop-style B2B shop	React SPA + ASP.NET JSON API	Magento 2, server-rendered HTML

	Sisters Point	Bestseller (MORE)	Hailys
Login	Username + password, Playwright browser	Email + OTP, so **cookies**	Customer no. + email + password, so **cookies**
Method	Playwright crawl of categories	Direct JSON API with `requests`	`requests` + HTML parsing
Scope	Whole site by category	**One favourite list** you build	Whole site by category group
Style key	Style-colour code, base stripped by exporter	`styleNumber`	`Artikelnummer` (one per colour item)
Sizes / stock	Size grid with stock	Not in API (one size)	Size matrix with stock (singles)
EAN	From the order grid	Not exposed	Not exposed

2.1 Sisters Point

```
        cp .env.example .env && cp config.example.yaml config.yaml   # add credentials
        python -m sp_scraper login
        python -m sp_scraper categories
        python -m sp_scraper scrape            # all categories, resumable
        python -m sp_scraper export            # rebuild files from
output/products.jsonl
```

Categories scraped: News, Noos, Shirts & Blouses, T-shirts. Output goes to `output/` and `output/by_category/`. See the main `README.md` for selectors and the `discover` debugging command.

2.2 Bestseller MORE

Bestseller has no public category pages. You scrape a **favourite list** (wishlist) that you build in the portal, so the list decides what you get.

1. Log in at more.bestseller.com. Open the favourite list. The list ID is in the URL: `...favouritelist?favouriteListId=<ID>`.
2. Copy the full `Cookie:` request header from DevTools → Network (not `document.cookie`, which misses the HttpOnly login cookie).
3. Save it: `python -m bs_scraper import-cookies "<cookie string>"`.
4. Scrape: `python -m bs_scraper scrape --list-id <ID> --output output/<name>`.

How the API works (found by reading the site's JS bundle):

- Every call needs three headers built from cookies: `__RequestVerificationToken` (from `CSRF-TOKEN`), `tabId` (from `sessionId`), `lastTabId` (from `sessionIdToUseForNewTab`). Without them the server returns the HTML app shell, not JSON.
- `GET /api/segmentation` returns the `segmentationId` the other calls need.
- `POST /api/favouritelist/page` with `pageNumber` (1-based), `pageSize`, `sortBy`, `groupingType: "Ungrouped"`, `favouriteListId`. Response: `lines[]` +

`pagingInformation.totalNumberOfItems`.
- Each line is a **style family**; its `masters[]` are the **colour variants**.

Known lists: `tbtayJwvFC` "NOOS AW first offer" (Vero Moda, 146 styles / 677 colours), `ZLs6MUPd7w` "NOOS AW" (ONLY, 55 styles / 339 colours). Always check the list's `totalCount` against the number scraped.

Use a separate `--output` folder per list so runs never overwrite each other.

2.3 Hailys

Login-gated Magento shop. Same cookie approach: copy the `Cookie:` header after logging in and save it as one line in `.auth/hailys_cookie.txt`.

```
        python -m hl_scraper scrape --fresh --workers 5     # output/hailys/
        python -m hl_scraper export                         # rebuild from
hl_products.jsonl
```

How it works:

1. **Crawl listings.** Group pages `/hailys/<group>.html` (17 groups), `/hailys-teens/<group>.html` (12), `/z-one/<group>.html` (5) and `/pos.html`. Pages are 24 items; paging is `?p=N`. The page text "Artikel 1 - 24 von N" gives the expected total, which the scraper checks.
2. **Dedupe.** URL is the key. `/nos.html`, `/new-in.html` and `/topseller.html` are crawled only to tag items already found. NOS items get `Season = NOS`. An item found twice is kept once.
3. **Scrape each product page.** Name (`Style - colour`), `Artikelnummer`, brand, material, size matrix, unit price, UVP and images.
4. **Map category** from the group (table below).

Singles vs lots: a product page can offer **Einzelartikel** (single pieces, with per-size stock in the "Verfügbar" row) and/or **Lots** (a fixed size run sold as a pack, for example XS1 S2 M3 L3 XL1 XXL1). Prices on the page are per piece. For lots the pack ratio and lot price go into `ProductText`, and stock is blank because the table shows pack quantities, not stock.

Category mapping (edit `CATEGORY_MAP` in `hl_scraper/extract.py`):

Catalogue category	Hailys groups
Footwear	Schuhe
Accessories	Accessoires, Schals/Tücher, Strümpfe/Strumpfhosen
Knitwear	Strickwaren
Outerwear	Jacken, Mäntel, Blazer, Westen
Tops	Shirts, Blusen, Hemden, Sweats
Bottoms	Hosen, Hosenanzüge, Jeans, Röcke
Dresses	Kleider, Sonstige (overalls)
NOS	Not a category: tagged in `Season`

Catalogue category	Hailys groups
Top Seller	Empty on the site at the time of writing

3. Adding a new supplier

1. **Look at it as a user.** Log in and click through the menu. Note every top-level group, how many items each has, and whether groups overlap (NOS, New In and Sale lists usually duplicate other categories).
2. **Find the cheapest data source,** in this order:
    1. A JSON API (DevTools → Network, filter Fetch/XHR, or read the JS bundle). Fastest and most complete.
    2. Server-rendered HTML (Magento, Shopify, WooCommerce): plain `requests` plus parsing.
    3. A browser (Playwright) only if the page is built entirely by JavaScript and has no API.
3. **Decide auth.** If the login uses OTP or a captcha, use the cookie method (Section 2.2 step 2) instead of automating the login.
4. **Write `<site>_scraper/`** with `extract.py` (page or JSON to `Product`) and `cli.py` (crawl, dedupe, scrape, export). Copy `hl_scraper` as the template.
5. **Pick the style key** so that one style = one `StyleCode` and colours sit inside it.
6. **Dry run on 3 products** (a simple one, a many-colour one, a special one such as a pack or shoe), compare with the website by eye.
7. **Full run, then validate** with the checklist below.

4. Validation checklist (run before sending files)

- Scraped styles equals the site's own count (list `totalCount`, or sum of category "N Artikel").
- `distinct ItemCode == number of rows` (no duplicate variants).
- No duplicate `StyleCode` + colour pair.
- Rows with empty wholesale price: 0. Rows with no image: 0, or explained.
- Category totals add up to the product total, with no uncategorised items.
- Spot-check 5 products against the website: name, colour, price, sizes, images.
- Shopify CSV: number of distinct handles equals the number of styles.

5. Troubleshooting

Symptom	Cause / fix
API or page returns the login or app shell HTML	Cookies expired or incomplete. Log in again and copy a fresh `Cookie:` header
`400 Bad Request` from an API	Wrong body shape. Copy the exact request payload from DevTools
Counts differ from the site	Check pagination (page size, 1-based vs 0-based) and the list's own total
Same item twice in the output	Item is in two categories; dedupe on URL and article number
App shows more styles than expected	`StyleCode` contains the colour; keep style and colour separate

Symptom	Cause / fix
Connection reset in a long run	Transient; the Hailys scraper retries 3 times. Lower `--workers` if it keeps happening

6. Known limits

- Bestseller: size, EAN and stock are not in the list API, so rows are one per colour with size "one size".
- Hailys: no per-size EAN on the page. The article number is the item code base. Care-label icons are not converted to text.
- Cookie-based sessions expire. Long runs may need the cookie refreshed.